

竞态条件脆弱性

版权所有 © 2017 杜文亮,保留所有权利。

允许个人使用。如需在课堂上使用这些习题,仅当该课程采用作者的著作作为教材时方可允许。所有其他用途均须事先征得作者的同意。

S7.1. 以下 Set-UID 程序是否存在竞态条件漏洞?

```
if (laccess( /etc/passwd , W_OK)) { /* 实际用户拥有写入权限 */ f =  
    open( /tmp/X , O_WRITE); write_to_file(f);  
  
} else { /* 真  
    实用户没有写入权限 */ fprintf(stderr, 权限被拒绝\n );  
}
```

S7.2. 假设我们开发了一个新的系统调用,名为 faccess(int fd, int mode),它与 access() 完全相同,区别在于要检查的文件由文件描述符 fd 指定。以下程序是否存在竞态条件问题?

```
int f = open( /tmp/x , O_WRITE); if (!faccess(f, W_OK))  
{ write_to_file(f) } else { close(f);  
  
}
```

S7.3. 在以下程序中,攻击者需要赢得多少个竞态条件?

```
int main()  
{  
    结构体 stat stat1, stat2;  
    int fd1, fd2;  
  
    if (access( /tmp/XYZ , O_RDWR)) { fprintf(stderr, 权限被拒绝  
        \n ); return -1;  
  
    } else fd1 = open( /tmp/XYZ , O_RDWR);  
  
    如果 (access( /tmp/XYZ , O_RDWR)) {  
        fprintf(stderr, 权限被拒绝\n ); return -1;  
  
    } else fd2 = open( /tmp/XYZ , O_RDWR);  
  
    程序随后检查 fd1 和 fd2 是否指向同一个文件,如果是,则程序会向 fd1 (或 fd2)写入数据。否则,程序不执行任何  
    操作。
```

```
    然后离开。
```

```
}
```

S7.4. 在 `open()` 系统调用中,它首先检查用户是否拥有访问目标文件的权限,然后再实际打开文件。这似乎是一种先检查后使用的模式。这种模式是否会导致竞态条件问题?

S7.5. 最小权限原则可以有效地防御本章讨论的竞态条件攻击。我们能否运用同样的原则来防御缓冲区溢出攻击?为什么?具体来说,在执行易受攻击的函数之前,我们禁用 `root` 权限;在易受攻击的函数返回之后,我们重新启用该权限。

S7.6. 以下由 `root` 用户拥有的 `Set-UID` 程序需要写入一个文件,但它需要确保该文件的所有者是当前用户。它使用 `stat()` 函数获取文件所有者的 ID,并将其与进程的实际用户 ID 进行比较。如果不匹配,程序将退出。请描述程序中是否存在竞态条件?如果存在,请解释如何利用该竞态条件。`stat()` 函数的手册可以在网上找到。

```
#include <stdio.h>
#include <sys/types.h> #include <sys/
stat.h> #include <unistd.h>

int main() {

    struct stat statbuf; uid_t real_uid;

    文件* fp;

    fp = fopen( "/tmp/XYZ", "a+" ); stat( "/tmp/XYZ",
    &statbuf);

    printf( "文件所有者的用户 ID:%d\n", statbuf.st_uid); printf( "进程的实际用户 ID:%d\n", getuid());

    // 检查文件是否属于用户 if (statbuf.st_uid == getuid()) {

        printf( "ID匹配,继续写入文件。\\n"); // 写入文件... if (fp) fclose(fp); } else { printf( "ID不匹配,退出。\\n"); if
        (fp) fclose(fp); return -1;

    }

    返回 0;
}
```

S7.7. 以下由 `root` 用户拥有的 `Set-UID` 程序需要写入文件,但它想要

确保文件归用户所有。程序使用 `fstat()` 获取文件所有者的 ID,并将其与进程的实际用户 ID 进行比较。如果不匹配,程序将退出。请描述程序中是否存在竞态条件?如果存在,请解释如何利用该竞态条件。`fstat()` 和 `fileno()` 的手册可以在网上找到。

```
#include <stdio.h>
#include <sys/types.h> #include <sys/
stat.h> #include <unistd.h>

int main() {

    struct stat statbuf; uid_t real_uid;

    文件* fp;

    fp = fopen( /tmp/XYZ , a+ ); fstat(文件号(fp),
    &statbuf);

    printf( 文件所有者的用户 ID:%d\n , statbuf.st_uid); printf( 进程的实际用户 ID:%d\n ,getuid());

    // 检查文件是否属于用户 if (statbuf.st_uid == getuid()) { printf( ID 匹配,继续写入文件。
    \n ); // 写入文件... if (fp) fclose(fp); } else { printf( ID 不匹配,退
    出\n ); if (fp) fclose(fp); return -1;

    }

    返回 0;
}
```

S7.8. 如果我们能够锁定一个文件,就可以通过在检查使用窗口期间锁定该文件来解决竞态条件问题,因为在此期间其他进程无法使用该文件。为什么我们不使用这种方法来解决本章讨论的竞态条件问题呢?

S7.9. 以下特权 Set-UID 程序是否存在竞态条件问题?如果存在,攻击窗口在哪里?请描述如何利用此竞态条件。

```
1 filename = /tmp/XYZ ; 2 fd = open (filename,  
O_RDWR); 3 status = access (filename, W_OK);  
  
...  
    (代码已省略)  
...  
10 如果 (状态 == ACCESS_ALLOWED) {  
11     写入文件(fd);  
12 } 否则 {  
13     fprintf(stderr, 权限被拒绝\n );  
14 }
```

S7.10. 请使用最小权限原则修复以下代码中的竞态条件问题程序。

```
如果 (access( /tmp/XYZ , W_OK) == ACCESS_ALLOWED) { f = open( /tmp/XYZ ,  
O_WRITE); write_to_file(f);  
  
} else  
    { fprintf(stderr, 权限被拒绝\n );  
}
```